Informatica® INFACore
November 2023

INFACore SDK for Python

Informatica INFACore INFACore SDK for Python
November 2023

© Copyright Informatica LLC 2022, 2023

This software and documentation are provided only under a separate license agreement containing restrictions on use and disclosure. No part of this document may be reproduced or transmitted in any form, by any means (electronic, photocopying, recording or otherwise) without prior consent of Informatica LLC.

U.S. GOVERNMENT RIGHTS Programs, software, databases, and related documentation and technical data delivered to U.S. Government customers are "commercial computer software" or "commercial technical data" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, duplication, disclosure, modification, and adaptation is subject to the restrictions and license terms set forth in the applicable Government contract, and, to the extent applicable by the terms of the Government contract, the additional rights set forth in FAR 52.227-19, Commercial Computer Software License.

Informatica, Informatica Cloud, Informatica Intelligent Cloud Services, PowerCenter, PowerExchange, and the Informatica logo are trademarks or registered trademarks of Informatica LLC in the United States and many jurisdictions throughout the world. A current list of Informatica trademarks is available on the web at https://www.informatica.com/trademarks.html. Other company and product names may be trade names or trademarks of their respective owners.

Portions of this software and/or documentation are subject to copyright held by third parties. Required third party notices are included with the product.

The information in this documentation is subject to change without notice. If you find any problems in this documentation, report them to us at infa_documentation@informatica.com.

Informatica products are warranted according to the terms and conditions of the agreements under which they are provided. INFORMATICA PROVIDES THE INFORMATION IN THIS DOCUMENT "AS IS" WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING WITHOUT ANY WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND ANY WARRANTY OR CONDITION OF NON-INFRINGEMENT.

Publication Date: 2023-12-01

Table of Contents

Preface 5

Informatica Resources 5

- Informatica Documentation 5
- Informatica Intelligent Cloud Services web site 5
- Informatica Intelligent Cloud Services Communities 5
- Informatica Intelligent Cloud Services Marketplace 6
- Data Integration connector documentation 6
- Informatica Knowledge Base 6
- Informatica Intelligent Cloud Services Trust Center 6
- Informatica Global Customer Support 6

Chapter 1: Introducing INFACore SDK for Python 7

INFACore SDK Reference for Python 7

Chapter 2: Setting up the runtime environment 8

Configure a remote agent 8

Configure a local agent 9

Chapter 3: Databricks integration with INFACore 10

Before you begin 10

Step 1. Install INFACore in Databricks Notebook 11

Step 2. Specify the remote agent engine details for Databricks Notebook 11

Sample notebook with code snippets 12

Restrictions 12

Chapter 4: Databricks integration with INFACore using Databricks Connect 13

Before you begin 13

- Set up the Databricks cluster 13
- Install Databricks Connect in your development environment 14
- Configure Databricks Connect 14

Procedure 15

- Step 1. Install INFACore in your development environment 15
- Step 2. Specify the remote agent engine details and the intermediate staging type 15

Sample notebook with code snippets 16

Chapter 5: Microsoft Fabric integration with INFACore 17

Before you begin 17

Step 1. Install INFACore in Microsoft Fabric 18

Step 2. Specify the remote agent engine details for Microsoft Fabric 18

Restrictions 19

Chapter 6: Reusing user-defined functions. 20

UDF scope and requirements. 20

Chapter 7: Reusing data pipelines. 21

Mapping template scope and requirements. 21

Chapter 8: Troubleshooting using INFACore log files. 22

Index. 23

Preface

Read *INFACore SDK for Python* to get familiar with INFACore Python SDK and understand how to use the Python SDK to manage data for your business from your development environment. See the INFACore SDK Reference for Python for the class definitions, methods, and parameters required to use the INFACore SDK to programmatically leverage INFACore.

Informatica Resources

Informatica provides you with a range of product resources through the Informatica Network and other online portals. Use the resources to get the most from your Informatica products and solutions and to learn from other Informatica users and subject matter experts.

Informatica Documentation

Use the Informatica Documentation Portal to explore an extensive library of documentation for current and recent product releases. To explore the Documentation Portal, visit https://docs.informatica.com.

If you have questions, comments, or ideas about the product documentation, contact the Informatica Documentation team at infa_documentation@informatica.com.

Informatica Intelligent Cloud Services web site

You can access the Informatica Intelligent Cloud Services web site at http://www.informatica.com/cloud. This site contains information about Informatica Cloud integration services.

Informatica Intelligent Cloud Services Communities

Use the Informatica Intelligent Cloud Services Community to discuss and resolve technical issues. You can also find technical tips, documentation updates, and answers to frequently asked questions.

Access the Informatica Intelligent Cloud Services Community at:

https://network.informatica.com/community/informatica-network/products/cloud-integration

Developers can learn more and share tips at the Cloud Developer community:

https://network.informatica.com/community/informatica-network/products/cloud-integration/cloud-developers

Informatica Intelligent Cloud Services Marketplace

Visit the Informatica Marketplace to try and buy Data Integration Connectors, templates, and mapplets:

https://marketplace.informatica.com/

Data Integration connector documentation

You can access documentation for Data Integration Connectors at the Documentation Portal. To explore the Documentation Portal, visit https://docs.informatica.com.

Informatica Knowledge Base

Use the Informatica Knowledge Base to find product resources such as how-to articles, best practices, video tutorials, and answers to frequently asked questions.

To search the Knowledge Base, visit https://search.informatica.com. If you have questions, comments, or ideas about the Knowledge Base, contact the Informatica Knowledge Base team at KB_Feedback@informatica.com.

Informatica Intelligent Cloud Services Trust Center

The Informatica Intelligent Cloud Services Trust Center provides information about Informatica security policies and real-time system availability.

You can access the trust center at https://www.informatica.com/trust-center.html.

Subscribe to the Informatica Intelligent Cloud Services Trust Center to receive upgrade, maintenance, and incident notifications. The Informatica Intelligent Cloud Services Status page displays the production status of all the Informatica cloud products. All maintenance updates are posted to this page, and during an outage, it will have the most current information. To ensure you are notified of updates and outages, you can subscribe to receive updates for a single component or all Informatica Intelligent Cloud Services components. Subscribing to all components is the best way to be certain you never miss an update.

To subscribe, go to https://status.informatica.com/ and click **SUBSCRIBE TO UPDATES**. You can then choose to receive notifications sent as emails, SMS text messages, webhooks, RSS feeds, or any combination of the four.

Informatica Global Customer Support

You can contact a Customer Support Center by telephone or online.

For online support, click **Submit Support Request** in Informatica Intelligent Cloud Services. You can also use Online Support to log a case. Online Support requires a login. You can request a login at https://network.informatica.com/welcome.

The telephone numbers for Informatica Global Customer Support are available from the Informatica web site at https://www.informatica.com/services-and-training/support-services/contact-us.html.

CHAPTER 1

Introducing INFACore SDK for Python

Use the INFACore SDK for Python in a Python environment to leverage the capabilities of INFACore.

The SDK is recommended if you are an experienced data scientist and you want to programmatically leverage INFACore in any Python environment for your data science projects.

To import the INFACore Python SDK, run the following code from your development environment:

```
import informatica.infacore as ic
```

You can invoke the INFACore SDK using the alias ic.

INFACore SDK Reference for Python

The INFACore SDK Reference for Python documents the INFACore SDK for Python functionality. This reference includes class definitions, methods, and parameters required to use the INFACore SDK to programmatically leverage INFACore.

The reference also covers examples that help you use the python SDK to consume, transform, and prepare data from any source

Use the *ic.help()* function to get the documentation for a specified module, class, and function.

For more information, see INFACore SDK Reference for Python.

CHAPTER 2

Setting up the runtime environment

You might want to use a local agent or a remote agent as the runtime environment depending on the development environment type where you use the INFACore Python SDK.

In cluster-based development environments, the installation of a remote agent might be necessary. On the other hand, in a local agent installation, the Secure Agent needs to be reinstalled locally each time the cluster goes down.

Configure a remote agent

You can configure a remote engine for cluster-based development environments. You can use an agent installed remotely in a Cloud computing service or a virtual machine. You need to specify the compute engine type as Remote and provide the runtime environment name in the INFACore SDK Python code.

Here is an example of the Python code snippet with the remote agent configuration:

```
user_config = {
    "login": {
        "username": "infacore",
        "password": "<password>",
        "env_type": "test",
        "url": "https://dmr-us.informaticacloud.com"
    },
    "compute_engine": {
        "type": "remote",
        "runtime_env_name": "<abc01.infa.com>",
        "install": False
      }
    }
}
```

Configure a local agent

You can configure an agent locally to your development environment. You need to specify the compute engine type as Local.

Here is an example of the Python code snippet with the local agent configuration:

```
user_config = {
    "login": {
        "username": "infacore",
        "password": "<password>",
        "env_type": "test",
        "url": "https://dmr-us.informaticacloud.com"
    },
    "compute_engine": {
        "type": "local",
        "install": True
      }
    }
}
```

CHAPTER 3

Databricks integration with INFACore

You can use INFACore in Databricks Notebook.

You can connect to various endpoints from Databricks Notebook to read from or write data. You also have the option to configure Databricks as an intermediate storage which helps you use INFACore seamlessly in Databricks Notebook. When you integrate INFACore with Databricks enabled for unity catalog, you can access and work with the metadata and data assets stored in the unity catalog directly from your Databricks Notebook.

For use cases that require staging of data, you can use the Databricks environment to stage the data temporarily. INFACore creates the Databricks native DataFrame for these use cases.

When you configure a Databricks environment for intermediate staging, you can use an agent installed remotely in a Cloud computing service or a virtual machine to stage the data in the Databricks environment.

You need to specify the compute engine type as Remote, provide the runtime environment name, and the Databricks Delta connection name in the Python SDK. INFACore uses the configured runtime environment to stage the data in Databricks using the specified Databricks Delta connection.

Before you begin

Before you install the INFACore Python SDK in Databricks Notebook, make sure you meet the following prerequisites:

- The agent is installed remotely. For instructions, see "Configure a remote agent" on page 8.
- You have a Databricks account and you have the required read and write permissions on the Databricks database.
- You have a Databricks Delta connection in your organization. If you do not have one already, create a Databricks Delta connection before you set up the Databricks environment.
  For more information about the properties required to create a Databricks Delta connection, see the *Connections for INFACore* documentation.

  For more information on how to create a Databricks Delta connection, see the *INFACore SDIK for Python Reference* documentation.

Step 1. Install INFACore in Databricks Notebook

Go to Informatica Marketplace and download and install the Informatica-infacore Python SDK package in your Databricks Notebook environment

For more information about the INFACore Python SDK installation steps, see the *INFACore Installation* and the *SDK Reference for Python* documentation.

For more information about how to install and configure the Python libraries in Databricks for the cluster-based environment, see the Databricks documentation.

Step 2. Specify the remote agent engine details for Databricks Notebook

After installing the SDK and logging in to INFACore from Databricks Notebook, configure Databricks as an intermediate storage.

Perform the following tasks in the INFACore SDK Python code to configure Databricks as an intermediate storage:

1. Enter the compute engine type as Remote and the runtime environment name.
2. Enter the ecosystem type and the configured Databricks Delta connection name.

Here is an example Python code snippet that demonstrates how to use INFACore with the remote agent details in a Databricks Notebook environment:

```
user_config = {
    "login": {
        "username": "infacore",
        "password": "<password>",
        "env_type": "test",
        "url": "https://dmr-us.informaticacloud.com"
    },
    "compute_engine": {
        "type": "remote",
        "runtime_env_name": "<abc01.infa.com>",
        "install": False
    },
    "ecosystem" : {
      "type": "Databricks Delta",
      "staging" :{
          "connection_name" : "Databricks_AWS"
      }
```

```
        }
    }
```

You can perform various operations or access functionalities that INFACore offers within your Databricks Notebook.

When you run the code, INFACore establishes a connection with Databricks Delta using the specified runtime environment. It uses the connection to stage the data in Databricks when you read from or write to the configured Databricks Delta endpoint.

For information on available INFACore functionalities and how to use specific methods and parameters, see the *SDK Reference for Python* documentation.

Sample notebook with code snippets

You can download a sample Databricks Notebook for reference from the following Informatica Knowledge Base article: KB 000205307

Restrictions

If the agent is installed remotely to your development environment, you cannot use the parse_unstructured_data() function.

CHAPTER 4

Databricks integration with INFACore using Databricks Connect

You can use INFACore in an integrated development environment that is compatible with Databricks such as JupyterLab, Visual Studio Code, or PyCharm.

Use the INFACore Python SDK code to leverage the INFACore functionality in your development environment, and then configure Databricks Connect from your environment to connect to a Databricks cluster and run the code remotely on that cluster. Databricks Connect is a client library for the Databricks runtime that runs jobs remotely on a Databricks cluster using Spark APIs.

For more information about how Databricks Connect works, see https://docs.databricks.com/dev-tools/databricks-connect.html.

Before you begin

Before you install the INFACore Python SDK in the development environment, make sure you meet the following prerequisites:

- You have a Databricks environment.
- Set up the Databricks clusters.
- Install and configure Databricks Connect in your development environment.
- Install the Secure Agent remotely. For instructions, see "Configure a remote agent" on page 8.
- You have a configured Databricks Delta connection in your organization to intermediately stage the data in Databricks. For more information about the properties required to create a Databricks Delta connection, see the *Connections for INFACore* documentation. For more information on how to create a Databricks Delta connection, see the *INFACore SDIK for Python Reference* documentation.

Set up the Databricks cluster

Set up the Databricks cluster for use with Databricks Connect. Databricks Connect runs your jobs remotely on a Databricks cluster using Spark APIs. The Databricks cluster must be using Databricks Runtime version 5.1 or later.

1. In the compute configuration of the Databricks cluster, go to the advanced options.

2. Edit the Spark configuration section and enter the following code snippet:

```
spark.databricks.service.server.enabled true
```

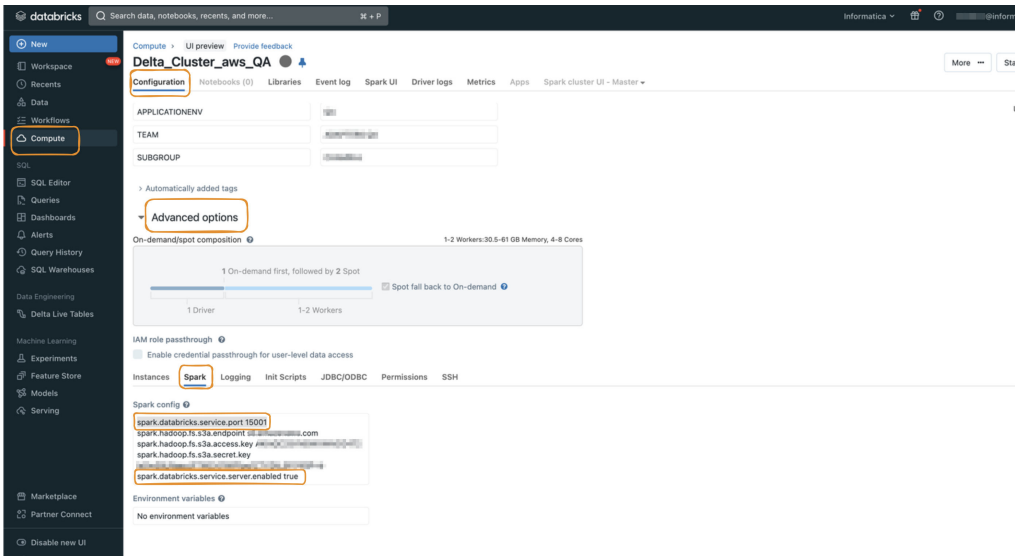
3. Enter the following code snippet, based on whether you want to use AWS Databricks or Azure Databricks:

   - To use AWS Databricks, enter the following code snippet:

     ```
     spark.databricks.service.port 15001
     ```

   - To use Azure Databricks, enter the following code snippet:

     ```
     spark.databricks.service.port 8787
     ```



4. Restart the cluster.

Install Databricks Connect in your development environment

Install the Databricks Connect library in your development environment.

1. Create a development environment. Ensure that your Python environment is compatible with the cluster version.
   For example, if you are using Anaconda, run the following code snippet to create a Databricks environment that is compatible with cluster version 3.x:

   ```
   conda create --name databricks python=3.8.10
   ```

2. Activate the development environment.
3. Install the Databricks Connect library.

   ```
   pip install -U databricks-connect==<databricks_connect_version>*
   ```

   The Databricks Connect version number needs to be compatible with the Databricks cluster version you are working with.

Configure Databricks Connect

Configure Databricks Connect and test the set up in your development environment before you can use it with INFACore.

1. From the command prompt, run the following command:

   ```
   databricks-connect configure
   ```

2. When prompted, provide the required information:

```
Databricks Host: <Databricks hostname>
*Databricks token: <Authentication token>
*Cluster ID* <Databricks cluster ID>
*Org ID: <Databricks ecosystem organization ID>
*Port: <Port number configured in the cluster Spark configuration>
Set new configuration values (leave input empty to accept default):
```

3. Test the Databricks Connect setup by running the following command:

```
databricks-connect test
```

Procedure

After you complete the prerequisites, install the INFACore SDK in the configured development environment. You need to specify the remote Secure Agent details and the Databricks ecosystem for intermediate storage.

After you configure these details, you can use Databricks Connect to connect to a remote Databricks cluster from your development environment to run INFACore jobs. You can perform various operations or access functionalities that INFACore offers within your development environment.
For information on available INFACore functionalities and how to use specific methods and parameters, see the INFACore SDK Reference for Python.

Step 1. Install INFACore in your development environment

Go to Informatica Marketplace and download and install the Informatica-infacore Python SDK package in your development environment, for example in VSCode, PyCharm, and JupyterLab.

For more information about the INFACore Python SDK installation steps, see the *INFACore Installation* and the *SDK Reference for Python* documentation.

Step 2. Specify the remote agent engine details and the intermediate staging type

In your development environment, set the remote agent runtime details. Also, set Databricks as an intermediate storage with Databricks Delta as the connection. INFACore uses the connection to stage the data in the development environment when you read from or write data to an endpoint.

Perform the following tasks in the INFACore SDK Python code in your development environment to set the remote agent and use Databricks as an intermediate storage. :

1. Enter the compute engine type as Remote and the runtime environment name.
2. Enter the ecosystem type and the configured Databricks Delta connection name.

Here is an example Python code snippet that demonstrates how to use INFACore with the remote agent details in a Databricks environment:

```
user_config = {
    "login": {
        "username": "infacore",
        "password": "<password>",
        "env_type": "test",
```

```
            "url": "https://dmr-us.informaticacloud.com"
        },
        "compute_engine": {
            "type": "remote",
            "runtime_env_name": "<abc01.infa.com>",
            "install": False
        },
        "ecosystem" : {
          "type": "Databricks Delta",
          "staging" :{
              "connection_name" : "Databricks_AWS"
          }
        }
    }
```

When you run the code, INFACore establishes a connection with Databricks Delta using the specified runtime environment. It uses the connection to stage the data in Databricks when you read from or write to the configured endpoint.

Sample notebook with code snippets

You can download a sample JupyterLab Notebook `.ipynb` file for reference from the following Informatica Knowledge Base article: KB 000205307

CHAPTER 5

Microsoft Fabric integration with INFACore

You can use INFACore in Microsoft Fabric. You can connect to various endpoints from Microsoft Fabric to read from or write data and also leverage other INFACore functionality. You also have the option to configure Microsoft Fabric as an intermediate storage which helps you use INFACore seamlessly in Microsoft Fabric Notebook.

For use cases that require staging of data, you can use the Microsoft Fabric environment to stage the data temporarily. INFACore creates the Microsoft Fabric pyspark DataFrame for these use cases. When you configure a Microsoft Fabric environment for intermediate staging, you can use an agent installed remotely in a Cloud computing service or a virtual machine to stage the data in the Microsoft Fabric environment.

You need to specify the compute engine type as Remote, provide the runtime environment name, and the Microsoft Fabric OneLake connection name in the Python SDK. INFACore uses the configured runtime environment to stage the data in Microsoft Fabric using the specified Microsoft Fabric OneLake connection.

Note: Effective in the November 2023 release, the Microsoft Fabric integration with INFACore functionality is available for preview. Preview functionality is supported for evaluation purposes but is unwarranted and is not supported in production environments or any environment that you plan to push to production. Informatica intends to include the preview functionality in an upcoming release for production use, but might choose not to in accordance with changing market or technical circumstances. For more information, contact Informatica Global Customer Support.

Before you begin

Before you install the INFACore Python SDK in Microsoft Fabric, make sure you meet the following prerequisites:

- The agent is installed remotely. For instructions, see "Configure a remote agent" on page 8.
- You have a Microsoft Fabric account and you have the required read and write permissions to Microsoft Fabric.
- You have a Microsoft Fabric OneLake connection in your organization. If you do not have one already, create a Microsoft Fabric OneLake connection before you set up the Microsoft Fabric environment. For more information about the properties required to create a Microsoft Fabric OneLake connection, see the *Connections for INFACore* documentation.

  For more information on how to create a Microsoft Fabric OneLake connection, see the *INFACore SDK for Python Reference* documentation.

Step 1. Install INFACore in Microsoft Fabric

Go to Informatica Marketplace and download and install the Informatica-infacore Python SDK package in your Microsoft Fabric Notebook environment

For more information about the INFACore Python SDK installation steps, see the *INFACore Installation* and the *SDK Reference for Python* documentation.

For more information about how to install and configure the Python libraries in Microsoft Fabric for the cluster-based environment, see the Microsoft Fabric documentation.

Step 2. Specify the remote agent engine details for Microsoft Fabric

After installing the SDK and logging in to INFACore from Microsoft Fabric, configure Microsoft Fabric as an intermediate storage.

Perform the following tasks in the INFACore SDK Python code to configure Microsoft Fabric as an intermediate storage:

1. Enter the compute engine type as Remote and the runtime environment name.
2. Enter the ecosystem type and the configured Microsoft Fabric OneLake connection name.

Here is an example Python code snippet that demonstrates how to use INFACore with the remote agent details in a Microsoft Fabric environment:

```
user_config = {
    "login": {
        "username": "infacore",
        "password": "<password>",
        "env_type": "test",
        "url": "https://dmr-us.informaticacloud.com"
    },
    "compute_engine": {
        "type": "remote",
        "runtime_env_name": "<abc01.infa.com>",
        "install": False
    },
    "ecosystem" : {
      "type": "Microsoft Fabric OneLake",
      "staging" :{
          "connection_name" : "OneLake_Prod"
      }
```

```
        }
    }
```

You can perform various operations or access functionalities that INFACore offers within your Microsoft Fabric Notebook.

When you run the code, INFACore establishes a connection with Microsoft Fabric using the specified runtime environment. It uses the connection to stage the data in Microsoft Fabric when you read from or write to the configured Microsoft Fabric endpoint.

For information on available INFACore functionalities and how to use specific methods and parameters, see the *SDK Reference for Python* documentation.

Restrictions

You cannot use the from_pandas() function in INFACore when you use INFACore in Microsoft Fabric.

CHAPTER 6

Reusing user-defined functions

If you have user-defined functions (UDFs) configured in your application, you can reuse the transformation logic from these UDFs in INFACore.

You can leverage mapplets from you your Informatica Intelligent Cloud Services organization as UDFs in INFACore. Get the list of configured UDFs from your organization and use these UDFs in INFACore and transform the data.

You can also chain UDFs to generate a data pipeline dynamically and run the pipeline. To chain the UDFs, set the UDF *cascade_fields* source attribute to *True* for a data pipeline in the Python SDK.

Note: The functionality of reusing UDFs in INFACore is available for preview. Preview functionality is supported for evaluation purposes but is unwarranted and is not supported in production environments or any environment that you plan to push to production. Informatica intends to include the preview functionality in an upcoming release for production use, but might choose not to in accordance with changing market or technical circumstances. For more information, contact Informatica Global Customer Support.

UDF scope and requirements

See the following scope and requirements before you reuse the transformation logic from UDFs configured in your Informatica Intelligent Cloud Services organization:

- You cannot use a UDF that is parameterized.
- If you set the UDF *cascade_fields* source attribute to *True* for a data pipeline, and a UDF in the pipeline is an active transformation that can change the number of rows that pass through the transformation, chaining of UDFs fails.
- You cannot use UDFs configured as source and target transformations in your Informatica Intelligent Cloud Services organization. You can use only those UDFs configured as other transformations, excluding source and target transformations, midstream in a data pipeline in INFACore. You can chain these UDFs either after the read operation or before the write operation.

CHAPTER 7

Reusing data pipelines

If you have parameterized mapping templates created in your Informatica Intelligent Cloud Services organization, you can reuse them as data pipelines in INFACore.

You can pass native data frames and other parameter values in these pipelines in INFACore and run multiple transformations using a single line of code. You can combine custom pipelines with native DataFrame functions, and achieve the required solution for your use case.

Mapping template scope and requirements

To reuse your Informatica Intelligent Cloud Services mapping templates in INFACore, consider the following guidelines:

- The mapping template must contain only one source and target. The template can have other transformations but cannot have multiple sources and targets.
- The source and target connections and objects in the template that you want to customize in INFACore needs to be parameterized. You can then define the values of these parameters in INFACore. For any other transformations configured in the template, you can chose to parameterize based on whether you want to use the transformation as such in INFACore or you want to customize it.
- When you use the parameterized source and target connections and objects, INFACore allows user inputs only for the source object and connection, but not for the target. INFACore uses the required intermediate staging connection and objects for the target by default.

CHAPTER 8

Troubleshooting using INFACore log files

You can monitor all your activities and status of the INFACore operations from the INFACore log files.

The log file also logs the methods invoked, REST call requests and responses, errors, and exceptions. Use the log file for failure analysis and to debug jobs in your organization.

You can get the details of the log file when you run the ic.help() function. The log file path lists in the code.

For example:

```
ic.help()
...
...
INFACORE SDK Version: 1.0.0
PYTHON Version: 3.7.1
OS Name: Windows
Log File Path: c:\Users\infacore\infacore.log
```

INDEX

C

Cloud Application Integration community
  URL 5
Cloud Developer community
  URL 5

D

Data Integration community
  URL 5

I

Informatica Global Customer Support
  contact information 6
Informatica Intelligent Cloud Services
  web site 5

M

maintenance outages 6

S

status
  Informatica Intelligent Cloud Services 6
system status 6

T

trust site
  description 6

U

upgrade notifications 6

W

web site 5